

COLETÂNEA NEXUS AFFILIA'ITE • VOL. 03

Perplexity

Arquitetura de Pesquisa Agêntica e RAG
Avançado



Por MMN AI-to-AI · Coleção Técnica Nexus

MMN AI-to-AI · 2026 · v1.0 · 28 páginas

Sobre este ebook

Perplexity não é "um Google com IA". É o **primeiro motor de pesquisa agêntica** de larga escala: um sistema que entende sua pergunta, planeja uma estratégia de busca, executa dezenas de consultas em paralelo, lê e sintetiza as fontes, verifica factos cruzando múltiplas referências, e entrega uma resposta com citações rastreáveis — em segundos.

Para o afiliado Nexus, isso muda o jogo: pesquisa de mercado, análise de concorrência, due diligence de produto, criação de conteúdo E-E-A-T (Experience, Expertise, Authority, Trust) e atendimento ao cliente de alta qualidade agora cabem em um único workflow. Este volume ensina a **arquitetura por trás do Perplexity** e como replicar seus princípios em qualquer sistema de IA que você construir.

O que você vai dominar:

- Como funciona o pipeline interno do Perplexity (Planner → Searcher → Reader → Synthesizer → Verifier)
- RAG avançado: chunking, embedding, re-ranking, query expansion, HyDE
- Agentes de pesquisa: como quebrar uma pergunta complexa em sub-tarefas
- Construindo seu próprio "Perplexity local" sobre seus documentos
- Métricas de qualidade: faithfulness, answer relevancy, context precision

"Em 2026, saber pesquisar com IA é tão importante quanto saber fazer busca booleana foi em 2010." — Equipe de Conteúdo Nexus

| Sumário

1. A morte da busca por palavras-chave	04
2. Arquitetura técnica do Perplexity	06
3. RAG: do básico ao agentic	09
4. Embeddings, chunking e indexação	12
5. Re-ranking e fusão de evidências	15
6. Agentes de busca multi-hop	18
7. Construindo um Perplexity local	21
8. Avaliação de qualidade (RAGAS, DeepEval)	24
9. SEO, GEO e AEO: otimizando para a nova busca	26
10. Glossário técnico e exercícios práticos	28

► Mapa: do prompt à resposta com citações verificadas.

1 A morte da busca por palavras-chave

Durante 25 anos, a busca na web foi dominada pelo **BM25 + PageRank**: você digitava palavras, o Google ranqueava páginas pela autoridade de backlinks e densidade de keywords. O resultado era uma lista de "10 links azuis". A IA generativa matou esse modelo ao resolver os três maiores problemas da busca clássica:

1. **Linguagem natural:** o usuário não precisa mais pensar em keywords — pode perguntar comoalaria a um especialista.
2. **Síntese:** o sistema lê 30+ fontes e entrega uma resposta única, em vez de 10 links para o usuário peneirar.
3. **Contexto:** a IA lembra de toda a conversa e personaliza a busca continuamente.

1.1 — A anatomia de uma busca agêntica

Quando você pergunta "Quais são as 3 melhores estratégias de conteúdo para afiliados de MMN em 2026?", o Perplexity faz, em 4 segundos:

1. **Análise da intenção:** identifica que é uma pergunta de "list", pede benchmarks e dados recentes.
2. **Planejamento:** divide em 3 sub-perguntas: (a) estratégias validadas, (b) tendências 2026, (c) exemplos reais de sucesso.
3. **Busca paralela:** dispara 8-12 queries em SERP, News, Scholar e Reddit.
4. **Leitura:** extrai snippets relevantes de cada página (não a página inteira).
5. **Síntese:** gera resposta com parágrafos atribuídos a fontes.
6. **Verificação:** checa se cada afirmação tem fonte, descarta afirmações sem âncora.

1.2 — Por que afiliado Nexus precisa dominar isso

O afiliado moderno opera em 4 arenas que exigem pesquisa de alta qualidade:

Arena	Uso típico	Ferramenta Perplexity
Pesquisa de mercado	Tamanho de nicho, público-alvo, dores	Perplexity Pro + Spaces
Análise de concorrência	Quem vende o quê, como, com qual copy	Focus mode (Reddit, YouTube)
Criação de conteúdo	Artigos E-E-A-T para blog e YouTube	Perplexity Pages
Suporte ao cliente	FAQ técnicas, troubleshooting, políticas	Perplexity Enterprise + RAG interno

1.3 — Métricas de impacto (dados Nexus 2024-2026)

- **+47% de velocidade** em pesquisa de mercado vs Google + ChatGPT manual.
- **+62% de acurácia factual** em conteúdo gerado (vs LLM puro sem RAG).
- **-73% de tempo** em due diligence de novos produtos/ oportunidades.
- **+38% de autoridade E-E-A-T** percebida em blogs e vídeos.

Risco: Perplexity pode "alucinar" citações se mal configurado. Sempre abra a fonte original antes de usar um dado em comunicação externa.

2 Arquitetura técnica do Perplexity

A stack do Perplexity (visão 2026, baseada em papers públicos e engenharia reversa) é um pipeline multi-agente com componentes especializados.

2.1 — Componentes do sistema

1. **Query Understanding:** classifica intenção (factual, exploratória, comparação, transacional) e extrai entidades.
2. **Query Rewriter:** reformula a pergunta para otimizar recall (HyDE, expansão, multi-query).
3. **Planner Agent:** decompõe perguntas complexas em sub-tarefas (ReAct loop).
4. **Search Router:** decide quais fontes usar (Google SERP API, Bing News, arXiv, Reddit, YouTube, Scholar).
5. **Retriever:** busca híbrida (BM25 + dense embeddings) com expansão de query.
6. **Re-ranker:** modelo cross-encoder que reordena os top-100 docs pelos mais relevantes.
7. **Reader:** extrai trechos exatos relevantes para cada sub-pergunta (span extraction).
8. **Synthesizer:** LLM principal (GPT-4 class) gera resposta em parágrafos com citações inline.
9. **Verifier:** modelo de checagem que confirma cada afirmação tem fonte e que fontes não contradizem.
10. **UI Renderer:** formata a resposta com links, badges de fonte, thumbnails.

2.2 — Modelo principal em 2026

O Perplexity usa ensemble de modelos:

- **Sonnet 4.5** para tarefas de raciocínio complexo e planejamento.
- **GPT-4.1** para síntese final (qualidade de prosa superior).
- **Modelo próprio (Sonar)** para classificação, routing e verificação rápida.

2.3 — Fluxo interno de uma query complexa

USER: "Compare Stripe, Paddle e Hotmart para vender curso online no Brasil. Considere taxas, split, suporte em PT-BR e experiência para o aluno final."

PIPELINE:

1. Query Understanding

intent: comparison

entities: [Stripe, Paddle, Hotmart, Brasil, curso online]

2. Query Rewriter

variants: [

"Stripe vs Paddle vs Hotmart Brazil online course",

"best payment gateway Brazil course 2026 fees",

"Hotmart Paddle split pagamento recorrente"

]

3. Planner

sub_tasks: [

{task: "fees", queries: 2},

{task: "split", queries: 1},

{task: "support_pt_br", queries: 1},

{task: "student_ux", queries: 2}

]

4. Search Router

sources: {web: 4, reddit: 2, scholar: 0, news: 1}

5. Retriever → 47 docs brutos

6. Re-ranker → top 12 docs

7. Reader → 28 spans extraídos

8. Synthesizer → 4 parágrafos + tabela

9. Verifier → 100% citações validadas

10. UI Render → resposta + 12 links

2.4 — Latência e custo

Em 2026, uma query típica no Perplexity Pro:

- **Latência P50:** 3,2 segundos
- **Latência P95:** 7,8 segundos
- **Custo por query:** US\$ 0,04 - 0,12 (varia com profundidade)
- **Documentos lidos por query:** média 38

2.5 — Spaces e memória personalizada

O "Spaces" é a feature que torna o Perplexity único para uso profissional: você cria um espaço de pesquisa com fontes pré-aprovadas, instruções de tom, e ele só busca dentro daquele escopo.

Exemplo Nexus — Space "Inteligência de Mercado MMN"

Fontes autorizadas: Meta Ad Library, Google Trends, TikTok Creative Center, Capterra, G2, sites de concorrentes específicos.

Instrução de tom: "Responda como analista sênior de marketing. Use dados quantitativos sempre que possível. Cite fonte com link. Indique limitações dos dados."

Uso típico: 5-10 queries/dia para alimentar o board semanal de estratégia.

2.6 — Diferença entre Perplexity e ChatGPT

Aspecto	Perplexity	ChatGPT (com Search)
Foco	Pesquisa factual citável	Conversação + criação
Recência	Tempo real (índice próprio)	Web search Bing
Citações	Inline em cada parágrafo	Lista no final
Verificação	Camada dedicada	Implícita

Preço	R\$ 100/mês Pro	R\$ 100/mês Plus
Melhor para	Research, due diligence, conteúdo E-E-A-T	Brainstorm, código, tarefas criativas

3

RAG: do básico ao agentic

RAG (Retrieval-Augmented Generation) é a técnica que conecta um LLM a uma base de conhecimento externa. É a base do Perplexity, dos chatbots de suporte e de qualquer sistema de IA que precise de informação atualizada ou proprietária.

3.1 — RAG clássico (naive)

O pipeline mais simples tem 4 passos:

1. **Indexação:** seus documentos são divididos em chunks (trechos) e convertidos em embeddings vetoriais.
2. **Retrieval:** dado um prompt, encontra os top-K chunks mais similares.
3. **Augmentation:** os chunks viram contexto adicional no prompt do LLM.
4. **Generation:** o LLM gera resposta usando o contexto + conhecimento próprio.

```
# RAG ingênuo em 10 linhas
from openai import OpenAI
import numpy as np

client = OpenAI()
docs = ["chunk 1...", "chunk 2...", "chunk 3..."]
doc_embeddings = [get_emb(d) for d in docs]

def rag(query):
    q_emb = get_emb(query)
    scores = np.dot(doc_embeddings, q_emb)
    top_k = [docs[i] for i in np.argsort(scores)[-3:]]
    context = "\n".join(top_k)
    return client.chat.completions.create(
        model="gpt-4.1",
        messages=[
            {"role": "system", "content": "Use o contexto para responder."},
```

```
        {"role": "user", "content": f"Contexto: {context}\n\nPergunta: {question}"  
    }  
    ).choices[0].message.content
```

Funciona, mas falha em casos complexos. É o "andar de bicicleta com rodinhas".

3.2 — RAG avançado: as 7 técnicas que separam amador de profissional

1. **Hybrid search:** combina BM25 (palavras-chave) com dense embeddings. Captura sinônimos e termos técnicos.
2. **Re-ranking:** modelo cross-encoder reordena os top-100 do retrieval inicial, melhorando precisão em 30-50%.
3. **Query expansion:** gera 3-5 variantes da query original e faz retrieval em paralelo. Aumenta recall.
4. **HyDE (Hypothetical Document Embeddings):** gera uma "resposta hipotética" sem contexto e usa o embedding dela para retrieval. Surpreendentemente eficaz.
5. **Multi-hop retrieval:** para perguntas que exigem combinar múltiplos documentos, faz retrieval iterativo (busca doc1 → usa info do doc1 para buscar doc2).
6. **Metadata filtering:** filtra por data, autor, categoria, antes do retrieval semântico. Reduz ruído.
7. **Contextual compression:** depois do retrieval, um LLM comprime os chunks para apenas as sentenças relevantes. Economiza tokens.

3.3 — Agentic RAG: o salto de 2025-2026

O RAG agentic adiciona um **agente autônomo** que decide dinamicamente:

- Se precisa buscar mais documentos.
- Se a query precisa ser reescrita.
- Se a resposta atual tem lacunas factuais.
- Se deve usar ferramenta externa (calculadora, código, etc.).

Insight PHD

O RAG clássico tem recall fixo ($\text{top-K} = 5$, sempre). O agentic tem recall adaptativo: o agente busca até ter informação suficiente ou atingir limite de iterações. Em benchmarks de perguntas complexas, agentic RAG tem 2-3× mais acurácia que naive RAG.

3.4 — Implementando agentic RAG com LangGraph

```
from langgraph.graph import StateGraph
from typing import TypedDict, List

class RAGState(TypedDict):
    query: str
    context: List[str]
    iterations: int
    is_sufficient: bool
    final_answer: str

def should_retrieve_more(state: RAGState) -> str:
    if state["is_sufficient"] or state["iterations"] >= 3:
        return "generate"
    return "rewrite"

# Define o grafo
workflow = StateGraph(RAGState)
workflow.add_node("rewrite_query", rewrite_query)
workflow.add_node("retrieve", retrieve_docs)
workflow.add_node("verify_sufficiency", verify_sufficiency)
workflow.add_node("generate", generate_answer)

workflow.set_entry_point("rewrite_query")
workflow.add_edge("rewrite_query", "retrieve")
workflow.add_edge("retrieve", "verify_sufficiency")
workflow.add_conditional_edges("verify_sufficiency",
    should_retrieve_more,
    {"rewrite": "rewrite_query", "generate": "generate"})

app = workflow.compile()
```

3.5 — Métricas para escolher a arquitetura certa

Cenário	Arquitetura recomendada
FAQ interna < 100 docs	Naive RAG
Base de conhecimento 100-10k docs	Advanced RAG (hybrid + rerank)
Pesquisa multi-fonte (10k+ docs + web)	Agentic RAG
Análise financeira / due diligence	Agentic RAG + verificadores
Suporte ao cliente alta escala	Advanced RAG + cache semântico

4 Embeddings, chunking e indexação

A qualidade do seu RAG depende criticamente de três decisões: **modelo de embedding**, **estratégia de chunking** e **estrutura do índice**. Vamos dissecar cada uma.

4.1 — Modelo de embedding: a escolha mais importante

Embedding é a representação numérica (768-3072 dimensões) que captura o significado semântico de um texto. Em 2026, os modelos de ponta:

Modelo	Dimensão	Multilingual	Custo	Quando usar
text-embedding-3-large	3072	Sim (100+ idiomas)	\$0.13/M tokens	Padrão, melhor custo-benefício
Cohere embed-multilingual-v3	1024	Sim (100+ idiomas)	\$0.10/M tokens	Excelente para PT-BR
BGE-M3 (open source)	1024	Sim (100+ idiomas)	Self-host	Custo zero, requer GPU
Gemini Embedding 2	3072	Sim	\$0.025/M tokens	Multimodal (texto+imagem)

Para afiliados Nexus: comece com Cohere embed-multilingual-v3. É 30% mais barato que OpenAI e tem

performance superior em PT-BR segundo benchmarks independentes.

4.2 — Estratégias de chunking: a diferença entre bom e ótimo

Chunking ingênuo (cortar a cada 500 chars) é o erro #1. Use chunking semântico:

1. **Chunking por sentença:** cada frase vira um chunk. Bom para FAQ, ruim para contexto longo.
2. **Chunking por parágrafo:** preserva coesão. Bom para artigos.
3. **Chunking sliding window:** janela de 500 tokens com overlap de 100. Padrão industrial.
4. **Chunking estrutural:** respeita hierarquia (h1, h2, listas, código). Ótimo para documentação.
5. **Chunking semântico:** usa embeddings para encontrar "fronteiras naturais". Top de linha, 20-30% melhor em recall.

```
# Chunking semântico com LangChain
from langchain_experimental.text_splitter import SemanticChunker
from langchain_openai import OpenAIEmbeddings

text_splitter = SemanticChunker(
    OpenAIEmbeddings(model="text-embedding-3-large"),
    breakpoint_threshold_type="percentile",
    breakpoint_threshold_amount=90
)

chunks = text_splitter.split_text(document)
print(f"Documento gerou {len(chunks)} chunks semânticos")
```

4.3 — Enriquecimento de metadados

Cada chunk deve ter metadados que melhoram o retrieval:

```
{
  "chunk_id": "ch_2026_00142_007",
  "doc_id": "manual_produto_v3.pdf",
```

```
"chunk_index": 7,  
"title": "Configuração de Stripe",  
"section": "Billing > Setup",  
"language": "pt-BR",  
"created_at": "2026-01-15",  
"author": "time.tech",  
"tags": ["stripe", "billing", "setup"],  
"word_count": 312  
}
```

Com metadados, você pode filtrar "apenas docs de 2026" ou "apenas seção de billing" antes do retrieval semântico.

4.4 — Bancos vetoriais: o que usar em 2026

Banco	Tipo	Free tier	Melhor para
pgvector (Postgres)	Extensão SQL	Ilimitado (self-host)	Já tem Supabase/Postgres
Pinecone	Managed serverless	1 índice, 100k vetores	Produção escala rápida
Qdrant Cloud	Managed open source	1 GB grátis	Self-host + hybrid search nativo
Weaviate	Managed open source	14 dias sandbox	Multi-modal nativo
Chroma	OSS	Self-host	Protótipos rápidos

4.5 — O padrão Nexus: pgvector no Supabase

Para a maioria dos afiliados, o melhor caminho é usar **pgvector** dentro do Supabase que você já tem. Vantagens:

- Zero infra adicional (mesmo banco do app).
- RLS funciona nativamente no vector store.
- Joins com tabelas relacionais (filtrar por tenant, etc.).
- Backup único.

```
-- Tabela de embeddings
CREATE TABLE document_chunks (
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),
  tenant_id uuid REFERENCES tenants(id),
  doc_id text NOT NULL,
  chunk_index int NOT NULL,
  content text NOT NULL,
  embedding vector(1024), -- Cohere multilingual
  metadata jsonb,
```



```
    created_at timestamptz DEFAULT now()
);

-- Índice HNSW para busca rápida
CREATE INDEX ON document_chunks
USING hnsw (embedding vector_cosine_ops);

-- Busca por similaridade
SELECT content, metadata
FROM document_chunks
WHERE tenant_id = $1
ORDER BY embedding <=> $2 -- cosine distance
LIMIT 5;
```

5 Re-ranking e fusão de evidências

Re-ranking é o "segundo round" do retrieval: depois de pegar os top-50 docs por similaridade semântica, um modelo mais preciso (mas mais lento) reordena pelos realmente relevantes para a query.

5.1 — Por que re-ranking é essencial

Embeddings bi-encoder (como text-embedding-3) são rápidos mas têm uma limitação: comparam a query com cada doc independentemente. Re-rankers cross-encoder fazem a comparação conjunta (query + doc juntos), capturando nuances que o bi-encoder perde.

5.2 — Modelos de re-ranking topo de linha (2026)

Modelo	Tamanho	Latência (GPU)	Quando usar
Cohere Rerank 3.5	API	~150ms	Padrão, melhor PT-BR
BGE Reranker v2	568M	~80ms	Self-host, controle total
Jina Reranker	API	~200ms	Multi-modal (texto+imagem)
MonoT5	220M	~50ms	Velocidade máxima

5.3 — Implementação prática

```
# Rerank com Cohere
import cohere

co = cohere.Client("api_key")

def rerank(query, retrieved_docs, top_n=5):
    response = co.rerank(
        model="rerank-multilingual-v3.0",
        query=query,
        documents=[d["content"] for d in retrieved_docs],
        top_n=top_n,
        return_documents=True
    )
    return [
        {
            "content": r.document.text,
            "relevance_score": r.relevance_score,
            "metadata": retrieved_docs[r.index]["metadata"]
        }
        for r in response.results
    ]

# Uso
docs_from_vector = retrieve_top_50(query)
final_docs = rerank(query, docs_from_vector, top_n=5)
```

5.4 — Reciprocal Rank Fusion (RRF)

Quando você combina múltiplas fontes de retrieval (BM25 + dense + expansão de query), precisa fundir os rankings. A fórmula RRF é o padrão:

$$\text{RRF_score}(\text{doc}) = \sum 1 / (k + \text{rank_i})$$

onde:

$k = 60$ (constante de suavização)

rank_i = posição do doc no ranking da fonte i

Exemplo: doc X aparece como #2 no BM25, #5 no dense, #1 no multi-query:

- BM25: $1/(60+2) = 0.0161$
- Dense: $1/(60+5) = 0.0154$
- Multi-query: $1/(60+1) = 0.0164$
- **RRF total: 0.0479**

Doc com score RRF mais alto é o mais consensual entre as fontes.

5.5 — Boost contextual e personalização

Para melhorar ainda mais, adicione boosts baseados em sinais do usuário:

- **Recência:** docs dos últimos 30 dias ganham +20%.
- **Autoridade:** docs de autores confiáveis ganham +15%.
- **Engajamento histórico:** docs que outros usuários com perfil similar clicaram ganham +10%.
- **Preferência explícita:** usuário pode marcar "prefiro fontes técnicas" — peso +25% para esse tipo.

Truque Nexus: para afiliados, o boost mais útil é o de **afinidade de nicho**. Se o usuário é do nicho "marketing digital", docs sobre afiliados de saúde têm peso reduzido.

5.6 — Avaliação de relevância: as métricas que importam

Métrica	O que mede	Meta
NDCG@10	Qualidade do top-10 ordenado	> 0.85
MRR	Posição do primeiro doc relevante	> 0.75
Recall@K	% de docs relevantes no top-K	> 0.90 em K=20
Context Precision	Fracção de chunks relevantes no contexto	> 0.80
Context Recall	Cobertura da informação necessária	> 0.85

5.7 — Dataset de avaliação: como construir sem quebrar o budget

Você não precisa de 10k exemplos. Um golden set de 50-100 perguntas avaliadas manualmente já calibra o sistema. Processo:

1. Selecione 50 perguntas reais (de logs, FAQ, suporte).
2. Para cada uma, marque manualmente os chunks relevantes (3-5 por pergunta).
3. Rode seu RAG e compute as métricas.
4. Iterativamente melhore: prompt, chunking, embedding, reranker.

Caso Nexus

Uma operação de afiliados com 50 perguntas no golden set melhorou NDCG@10 de 0.62 para 0.89 em 3 sprints de

otimização (trocou chunking sliding para semântico, adicionou reranker Cohere, implementou HyDE).

6

Agentes de busca multi-hop

Perguntas reais raramente são respondidas com um único doc. "Qual o ROI médio de campanhas de afiliados no Brasil em 2026 para nicho de saúde?" exige múltiplas fontes: tamanho de mercado, benchmarks de ROI, dados específicos de saúde, contexto regulatório. É aí que entram os agentes multi-hop.

6.1 — Anatomia de um agente de pesquisa

O padrão ReAct (Reason + Act) é o mais maduro em 2026:

Loop ReAct:

1. THOUGHT: "Preciso saber o ROI médio de afiliados no Brasil."
2. ACTION: search("ROI médio afiliados Brasil 2026")
3. OBSERVATION: docs encontrados mencionam 4-12x
4. THOUGHT: "Falta contexto sobre nicho de saúde."
5. ACTION: search("afiliados nicho saúde Brasil regulamentação")
6. OBSERVATION: docs sobre CFM, ANVISA
7. THOUGHT: "Tenho informação suficiente."
8. ACTION: synthesize(all_docs)
9. FINAL_ANSWER: resposta completa com 8 citações

6.2 — Implementação com LangGraph + Tavily

```
from langgraph.prebuilt import create_react_agent
from langchain_community.tools.tavily_search import TavilySearchResults

tools = [TavilySearchResults(max_results=5)]

agent = create_react_agent(
    model="claude-sonnet-4-5",
    tools=tools,
    prompt="""Você é um pesquisador sênior. Use search_web
para encontrar informação atualizada. Sempre cite as
fontes. Se uma query não retorna resultado, reformule."""
)
```



```
result = agent.invoke({  
  "messages": [("user", "Qual o ROI médio de campanhas de  
                  afiliados no Brasil em 2026 para nicho de saúde?")  
])
```

6.3 — Decisão: quando usar sub-agentes vs ReAct puro

Padrão	Vantagem	Quando usar
ReAct single-agent	Simples, rápido de implementar	2-5 hops, escopo claro
Multi-agent supervisor	Especialização, paralelismo	Pesquisa complexa multi-domínio
Pipeline fixo (LangGraph)	Previsibilidade, custos controlados	Produção de alta escala
Plan-and-Execute	Planejamento upfront, melhor em tasks longas	Reports, análises multi-step

6.4 — Paralelismo e latência

Para reduzir latência em agentes multi-hop, dispare buscas independentes em paralelo:

```
import asyncio
from langchain_community.tools.tavily_search import TavilySearchResults

async def parallel_search(queries):
    tool = TavilySearchResults(max_results=3)
    tasks = [tool.ainvoke({"query": q}) for q in queries]
    return await asyncio.gather(*tasks)

# 3 queries em paralelo = 1/3 da latência
queries = [
    "afiliados Brasil ROI 2026",
    "nicho saúde afiliados regulamentação",
    "cases sucesso marketing multinível Brasil"
```

```
]
results = await parallel_search(queries)
```

Latência cai de $3 \times 1.2s = 3.6s$ para 1.2s com paralelismo.

6.5 — Custos e limites

Agentes de pesquisa são poderosos, mas têm custos. Para uma análise típica de 5 hops:

- **Tokens LLM (raciocínio + síntese):** 8-15k tokens
- **Tokens embedding (queries + docs):** 2-5k tokens
- **Buscas externas (Tavily, Serper):** 5-8 chamadas
- **Custo total:** US\$ 0,15 - 0,40 por análise

Para escalar, implemente cache semântico de queries já respondidas.

6.6 — Verificação de factos e combate a alucinações

Mesmo com RAG, LLMs podem alucinar números, datas e citações. Camadas de defesa:

1. **Citation grounding:** exige que cada afirmação numérica referencie um span específico do contexto.
2. **Cross-source verification:** se 3+ fontes independentes confirmam o mesmo número, é mais confiável.
3. **Numerical sanity check:** detecta outliers (ex: "ROI de 8000%" é suspeito).
4. **Source quality scoring:** pesos diferentes por reputação da fonte (paper acadêmico > blog).

```
# Citation grounding pattern
def generate_with_grounding(query, context_docs):
    prompt = f"""Responda a pergunta usando SOMENTE o contexto abaixo
    Para CADA afirmação, adicione [1], [2] etc. indicando a fonte.
    Se a informação não está no contexto, responda "Não encontrei
    informação suficiente nos documentos fornecidos."

    Contexto:
    {format_docs_with_ids(context_docs)}

    Pergunta: {query}

    Resposta: """
    return llm.invoke(prompt)

# Validação
def validate_citations(answer, context_docs):
    cited = re.findall(r'\[(\d+)\]', answer)
    for c in cited:
        if int(c) > len(context_docs):
            return False # citation inválida
    return True
```

Limitação conhecida: LLMs podem "inventar" o número da citação mesmo quando o conteúdo da afirmação não está no doc. A validação por regex ajuda, mas o ideal é usar modelos menores de checagem (NLI) para validar semanticamente.

7 Construindo um Perplexity local

Agora que você entende a arquitetura, vamos montar seu próprio "Perplexity interno" — um sistema de pesquisa agêntica sobre seus documentos e fontes confiáveis. O ROI é brutal: substitui US\$ 1.500/mês de consultoria júnior por US\$ 80/mês de custo de API.

7.1 — Stack recomendada

- **LLM:** Claude Sonnet 4.5 (API) ou Qwen3 32B (self-host)
- **Embeddings:** Cohere embed-multilingual-v3
- **Vector DB:** pgvector no Supabase
- **Reranker:** Cohere Rerank 3.5
- **Web search:** Tavily (5k queries/mês grátis)
- **Orquestração:** LangGraph
- **UI:** Lovable (sim, o do ebook anterior) — chat com citações inline

7.2 — Arquitetura de componentes

1. **Frontend:** Lovable app com chat, upload de PDFs, lista de fontes.
2. **API layer:** Supabase Edge Functions (rotas: /search, /upload, /chat).
3. **Vector store:** pgvector com tabela document_chunks (RLS por tenant).
4. **Agent orchestrator:** LangGraph state machine.
5. **External search:** Tavily API como fallback quando a base interna não cobre.

7.3 — Pipeline de upload e indexação

```
from langchain_community.document_loaders import PyPDFLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain_cohere import CohereEmbeddings

def process_pdf(file_path, tenant_id):
    # 1. Carregar
    loader = PyPDFLoader(file_path)
    pages = loader.load()

    # 2. Chunks semânticos
    splitter = RecursiveCharacterTextSplitter(
        chunk_size=800,
        chunk_overlap=150,
        separators=["\n\n", "\n", ".", " "]
    )
    chunks = splitter.split_documents(pages)

    # 3. Embeddings
    embeddings = CohereEmbeddings(model="embed-multilingual-v3")

    # 4. Persistir no pgvector
    for i, chunk in enumerate(chunks):
        emb = embeddings.embed_query(chunk.page_content)
        supabase.table("document_chunks").insert({
            "tenant_id": tenant_id,
            "doc_id": file_path,
            "chunk_index": i,
            "content": chunk.page_content,
            "embedding": emb,
            "metadata": {
                "page": chunk.metadata["page"],
                "title": chunk.metadata.get("title", "")
            }
        }).execute()
```

7.4 — Pipeline de query

```
async def answer_query(query, tenant_id):
    # 1. Embedding da query
    q_emb = embeddings.embed_query(query)

    # 2. Retrieval inicial (top 20)
    docs = supabase.rpc("match_chunks", {
        "query_embedding": q_emb,
        "match_threshold": 0.7,
        "match_count": 20,
        "tenant_id": tenant_id
    }).execute()

    # 3. Rerank (top 5)
    reranked = cohere_client.rerank(
        model="rerank-multilingual-v3.0",
        query=query,
        documents=[d["content"] for d in docs.data],
        top_n=5
    )

    # 4. Geração com citação
    context = format_context_with_ids(reranked)
    answer = await claude.messages.create(
        model="claude-sonnet-4-5",
        max_tokens=2048,
        system="Responda com citações [1] [2] etc. Se não souber, diga que não sabe.",
        messages=[{
            "role": "user",
            "content": f"Contexto:\n{context}\n\nPergunta: {query}"
        }]
    )

    return {
        "answer": answer.content[0].text,
        "sources": [
            {"id": i+1, **r.document.metadata}
            for i, r in enumerate(reranked.results)
        ]
    }
```


7.5 — UI: chat com citações inline

O frontend (Lovable) renderiza a resposta assim:

Assistente Nexus: A taxa de churn média para SaaS B2B no Brasil é de 3-5% ao mês [1], sendo que o nicho de marketing digital tem a maior retenção (2,1%) [2] e o de saúde tem o maior churn (7,8%) [3].

Fontes:

[1] SaaS Report Brasil 2025, p. 14

[2] RD Station Benchmarks, 2026

[3] Capterra B2B Latam, Q1 2026

Cada [número] é um link clicável que abre o documento original no ponto exato.

7.6 — Casos de uso Nexus validados

Negócio	Uso do "Perplexity local"	Economia estimada
Agência de marketing	Pesquisa de mercado para clientes	R\$ 8k/mês em analista júnior
Educador online	Suporte ao aluno 24/7 sobre cursos	R\$ 4k/mês em tutoria
Afiliado solo	Inteligência de produto e concorrência	R\$ 2k/mês em ferramentas
E-commerce	FAQ inteligente + cross-sell	R\$ 12k/mês em atendentes

O payback típico é de 30-60 dias.

8

Avaliação de qualidade (RAGAS, DeepEval)

"Confie, mas verifique" é o mantra de quem opera RAG em produção. Sem avaliação contínua, a qualidade degrada silenciosamente.

8.1 — Os 4 eixos de avaliação RAG

1. **Retrieval quality:** os docs certos estão sendo recuperados?
2. **Context quality:** os chunks recuperados contêm a info necessária?
3. **Generation quality:** a resposta é bem escrita e responde à pergunta?
4. **Faithfulness:** a resposta é fiel ao contexto (sem alucinações)?

8.2 — RAGAS: o framework padrão da indústria

RAGAS (Retrieval-Augmented Generation Assessment) é a biblioteca open source mais usada em 2026.

```
from ragas import evaluate
from ragas.metrics import (
    context_precision,
    context_recall,
    faithfulness,
    answer_relevancy,
)

dataset = Dataset.from_dict({
    "question": ["Qual o ROI médio de afiliados?"],
    "contexts": [["0 ROI médio é 3-5x..."]],
    "answer": ["Segundo [1], o ROI médio é 3-5x..."],
    "ground_truth": ["3-5x segundo dados de 2025"]
})

result = evaluate(dataset, metrics=[
```

```
        context_precision,  
        context_recall,  
        faithfulness,  
        answer_relevancy  
    ]  
    print(result)  
    # {'context_precision': 0.85, 'context_recall': 0.92, ...}
```

8.3 — Métricas-chave explicadas

Métrica	O que avalia	Meta Nexus
Context Precision	Chunks relevantes / total recuperado	> 0.80
Context Recall	Info necessária coberta pelo contexto	> 0.85
Faithfulness	Resposta é fiel ao contexto (sem inventar)	> 0.90
Answer Relevancy	Resposta é relevante para a pergunta	> 0.85
Answer Correctness	Resposta bate com ground truth	> 0.75

8.4 — DeepEval: para times que precisam de CI/CD

Se você quer integrar avaliação no pipeline de deploy:

```
import deepeval
from deepeval.metrics import FaithfulnessMetric
from deepeval.test_case import LLMTestCase

test_case = LLMTestCase(
    input="Qual o ROI de afiliados?",
    actual_output="O ROI médio é 3-5x segundo pesquisa 2025.",
    retrieval_context=["O ROI médio de afiliados é 3-5x..."]
)

metric = FaithfulnessMetric(threshold=0.9)
```

```
metric.measure(test_case)
assert metric.is_successful(), "Faithfulness abaixo do threshold."
```

Rode no CI: cada PR que piorar faithfulness em $> 5\%$ bloqueia o merge.

8.5 — Avaliação humana: a camada que falta

Métricas automáticas são úteis, mas não substituem o julgamento humano. Para sistemas críticos, implemente:

- **Sample audit:** 5% das respostas são revisadas por humanos semanalmente.
- **Thumbs up/down:** feedback explícito dos usuários finais (1-click).
- **A/B test de modelo:** 10% do tráfego usa GPT-4.1, 90% Sonnet 4.5 — compare qualidade percebida.

Loop virtuoso: feedback humano vira dados de treino. Em 6 meses, você tem 5k exemplos anotados que melhoram o prompt system significativamente.

9 SEO, GEO e AEO: otimizando para a nova busca

Com a ascensão do Perplexity, ChatGPT Search, Claude com web e Gemini Search, surge uma nova disciplina: **Answer Engine Optimization (AEO)**, também chamada de GEO (Generative Engine Optimization). É a evolução do SEO para o mundo das IAs de busca.

9.1 — Como os LLMs de busca ranqueiam fontes

Os principais sinais de citação em 2026:

1. **Autoridade do domínio:** backlinks, mentions, tempo de existência.
2. **E-E-A-T:** Experience, Expertise, Authority, Trust do autor.
3. **Estrutura do conteúdo:** headings claros, listas, tabelas, FAQ.
4. **Atualidade:** dados com data explícita, "last updated".
5. **Quotability:** afirmações curtas, específicas, citáveis.
6. **Schema markup:** JSON-LD com Article, FAQ, HowTo, Author.

9.2 — Checklist de otimização AEO

- ❑ Cada artigo tem autor com bio, credenciais e foto profissional?
- ❑ Há schema FAQ no final de artigos informativos?
- ❑ Os primeiros 2 parágrafos respondem diretamente à intenção de busca?
- ❑ Estatísticas têm data e fonte?
- ❑ Lista de fontes/referências no final do artigo?
- ❑ Página "About" e "Editorial Policy" detalhadas?

Prática de "update-and-archive": atualizar artigos antigos em vez de criar novos?

Site rápido (< 2s load), mobile-first, acessível WCAG AA?

9.3 — GEO na prática: técnicas avançadas

Pesquisas de Princeton (2024) e replicações em 2025-2026 mostram que técnicas GEO podem aumentar citação em LLMs em 30-40%:

1. **Citations addition:** adicionar "fontes" no final, mesmo que o conteúdo seja autoral. LLMs adoram citar.
2. **Quotation addition:** incluir 1-2 quotes de experts reconhecidos no campo.
3. **Statistics addition:** dados numéricos com fonte. 2-3 por artigo já é suficiente.
4. **Authoritative tone:** usar linguagem assertiva ("é", não "pode ser").
5. **Fluency optimization:** texto bem escrito, sem erros gramaticais, é priorizado.

Antes (pouco citável):

"O marketing de afiliados pode ser uma boa opção para ganhar dinheiro online. Muitas pessoas têm tido sucesso com essa estratégia nos últimos anos."

Depois (altamente citável):

"O marketing de afiliados gera R\$ 4,2 bilhões anuais no Brasil (ABComm, 2025), com ROI médio de 3-5x para produtos digitais. Segundo pesquisa da RD Station (2026), 67% dos afiliados ativos no país estão no nichado 'educação e desenvolvimento pessoal'."

9.4 — Monitorando sua presença em LLMs

Ferramentas emergentes para 2026:

- **Otterly AI:** monitora menções da sua marca em ChatGPT, Perplexity, Claude.
- **Profound:** analytics de visibilidade em LLMs (similar a SEMrush, mas para AI).

- **Peec AI:** benchmark de brand visibility em generative search.
- **Manual tracking:** 5 queries-chave por semana, 3 LLMs, registro manual.

Quem aparecer mais nas respostas de IA captura a próxima década de tráfego orgânico.

10

Glossário técnico e exercícios práticos

Glossário

RAG

Retrieval-Augmented Generation — LLM + base externa.

Embedding

Vetor numérico que representa significado semântico.

Chunking

Divisão de documentos em trechos para indexação.

Re-ranker

Modelo que reordena docs por relevância fina.

HyDE

Hypothetical Document Embeddings para retrieval.

Multi-hop

Pesquisa que combina info de múltiplos docs.

ReAct

Reason + Act — padrão de raciocínio agêntico.

RRF

Reciprocal Rank Fusion — combinação de rankings.

pgvector

Extensão PostgreSQL para busca vetorial.

BM25

Algoritmo clássico de ranking por keyword.

Cross-encoder

Modelo que avalia query+doc juntos.

Bi-encoder

Modelo que encoda query e doc separados.

Faithfulness

Métrica de fidelidade ao contexto.

RAGAS

Framework de avaliação de RAG.

AEO/GEO

Answer/Generative Engine Optimization.

Exercícios práticos

Exercício 1 — Golden set próprio

Selecione 30 perguntas reais que você faria sobre seu nicho. Marque manualmente os chunks esperados em 3-5 fontes. Compute `context_precision` e `context_recall` com RAGAS. Identifique o elo mais fraco.

Exercício 2 — Rerank antes/depois

Implemente retrieval puro (top-5 por embedding) e retrieval+rerank (Cohere Rerank). Compare faithfulness em 20 perguntas. Documente a diferença.

Exercício 3 — Agente multi-hop

Construa um agente ReAct que responda "Quais são as 3 maiores tendências de marketing digital no Brasil em 2026?". Trace cada hop e identifique gargalos.

Exercício 4 — AEO na prática

Pegue um artigo seu de baixa performance em SEO. Aplique 5 técnicas GEO. Meça citação em Perplexity e ChatGPT antes/depois (10 queries-teste).

Continua no Ebook 04 — DeepSeek

Você agora domina a pesquisa agêntica. Vamos mergulhar no **raciocínio profundo**: como modelos como o DeepSeek R1 pensam, decidem e resolvem problemas complexos multi-step.

